

大模拟时代

时间：2024 年 10 月 26 日 14:30 ~ 18:30

题目名称	Pegman	负载均衡	Family Tree B	文件压缩
题目类型	standard	standard	standard	standard
目录	pegman	load	tree	zip
可执行文件名	pegman	load	tree	zip
输入文件名	pegman.in	load.in	tree.in	zip.in
输出文件名	pegman.out	load.out	tree.out	zip.out
每个测试点时限	5s ~ 10s	1 秒	1 秒	1 秒
内存限制	1 GB	128 MB	256 MB	125 MB
测试点数目	10	10	10	10
测试点是否等分	是	是	是	是

提交源程序文件名

对于 C++ 语言	pegman.cpp	load.cpp	tree.cpp	zip.cpp
-----------	------------	----------	----------	---------

编译选项

对于 C++ 语言	- O2 - std=c++14 - static
-----------	---------------------------

注意事项（请仔细阅读）

1. 文件名（程序名和输入输出文件名）必须使用英文小写。
2. `main` 函数的返回值类型必须是 `int`，程序正常结束时的返回值必须是 0。
3. 提交的程序代码文件的放置位置请参考各省的具体要求。
4. 因违反以上三点而出现的错误或问题，申诉时一律不予受理。
5. 若无特殊说明，结果的比较方式为全文比较（过滤行末空格及文末回车）。
6. 选手提交的程序源文件必须不大于 100KB。
7. 程序可使用的栈空间内存限制与题目的内存限制一致。
8. 全国统一评测时采用的机器配置为：Intel(R) Core(TM) i7-8700K CPU @3.70GHz，内存 32GB。上述时限以此配置为准。
9. 只提供 Linux 格式附加样例文件。
10. 评测在当前最新公布的 NOI Linux 下进行，各语言的编译器版本以此为准。

Pegman (pegman)

P13224 [GCJ 2015 #2] Pegman

【题目描述】

在使用 Google 街景时，你可能曾经拖动并放下过角色 Pegman。今天，一个调皮的用户要把 Pegman 放在一个 R 行 C 列的单位格矩形网格中的某个格子里。这个网格中的每个格子可能是空白的，也可能标有一个箭头，箭头指向四个可能的方向之一：上、右、下或左。

当 Pegman 被放在一个格子上时，如果该格子是空白的，Pegman 会永远静止不动。然而，如果该格子上有一个箭头，Pegman 会开始朝那个方向行走。在行走过程中，每当他遇到空白格子时，他会继续保持当前方向前进；但每当他遇到另一个箭头时，他会转向该箭头指示的方向，然后继续行走。

你知道 Pegman 可能会一直在网格上快乐地循环行走，但也有可能 Pegman 会走出网格的边界！你可以通过改变一个或多个箭头的方向来防止这种情况发生，从而拯救他。（每个箭头的方向只能更改为另外三种可能的方向；只能更改箭头，不能添加或移除箭头。）

你要求出，最少需要更改多少个箭头的方向，才能确保无论 Pegman 最初被放在网格的哪个位置，他都不会走出网格边界？如果无论怎么更改箭头都无法保证这一点，则输出 IMPOSSIBLE。

【输入格式】

输入的第一行为测试用例数 T 。接下来有 T 组测试数据。每组测试数据的第一行为两个用空格分隔的整数 R 和 C 。接下来有 R 行，每行有 C 个字符，描述网格的每个格子，每个字符为以下之一：

- 1 . 句点 = 无箭头
- 2 ^ 上箭头
- 3 > 右箭头
- 4 v 下箭头
- 5 < 左箭头

【输出格式】

对于每组测试数据，输出一行，格式为 "Case # x : y "，其中 x 为测试用例编号（从 1 开始）， y 为最少需要更改的箭头数量，或者如果无论如何都无法保证 Pegman 不会走出网格，则输出 IMPOSSIBLE。

【输入输出样例 #1】

输入 #1

```
1 4
2 2 1
3 ^
4 ^
5 2 2
6 >v
7 ^<
8 3 3
9 ...
10 .^ .
11 ...
12 1 1
13 .
```

输出 #1

```
1 Case #1: 1
2 Case #2: 0
3 Case #3: IMPOSSIBLE
4 Case #4: 0
```

【说明/提示】**样例解释**

在第 1 组样例中，无论 Pegman 被放在哪里，他都一定会走出网格的上边界。你可以通过将最上面的箭头改为向下，从而让 Pegman 在这两个箭头之间来回循环，避免走出网格。

在第 2 组样例中，无论 Pegman 被放在哪里，他都会顺时针绕着网格循环行走，无需更改任何箭头。

在第 3 组样例中，调皮的用户可能会把 Pegman 放在网格中央的上箭头上，这样他会开始行走并最终走出网格的上边界。更改这个箭头的方向也无济于事：他只会从别的边界走出去。

在第 4 组样例中，唯一的起始格子是空白格，Pegman 会永远静止，不会有危险。

数据范围

- $1 \leq T \leq 100$ 。

小数据集（5 分）

- 时间限制：5 秒。
- $1 \leq R, C \leq 4$ 。

大数据集（10 分）

- 时间限制：10 秒。
 - $1 \leq R, C \leq 100$ 。
- 由 ChatGPT 4.1 翻译

负载均衡 (load)

P8755 [蓝桥杯 2021 省 AB2] 负载均衡

【题目描述】

有 n 台计算机，第 i 台计算机的运算能力为 v_i 。

有一系列的任务被指派到各个计算机上，第 i 个任务在 a_i 时刻分配，指定计算机编号为 b_i ，耗时为 c_i 且算力消耗为 d_i 。如果此任务成功分配，将立刻开始运行，期间持续占用 b_i 号计算机 d_i 的算力，持续 c_i 秒。

对于每次任务分配，如果计算机剩余的运算能力不足则输出 -1 ，并取消这次分配，否则输出分配完这个任务后这台计算机的剩余运算能力。

【输入格式】

输入的第一行包含两个整数 n, m ，分别表示计算机数目和要分配的任务数。

第二行包含 n 个整数 $v_1, v_2, \dots, v_n$ ，分别表示每个计算机的运算能力。

接下来 m 行每行 4 个整数 a_i, b_i, c_i, d_i ，意义如上所述。数据保证 a_i 严格递增，即 $a_i < a_{i+1}$ 。

【输出格式】

输出 m 行，每行包含一个数，对应每次任务分配的结果。

【输入输出样例 #1】

输入 #1

```
1 2 6
2 5 5
3 1 1 5 3
4 2 2 2 6
5 3 1 2 3
6 4 1 6 1
7 5 1 3 3
8 6 1 3 4
```

输出 #1

```
1 2
2 -1
3 -1
```

4	1
5	-1
6	0

【说明/提示】**【样例说明】**

时刻 1, 第 1 个任务被分配到第 1 台计算机, 耗时为 5, 这个任务时刻 6 会结束, 占用计算机 1 的算力 3。

时刻 2, 第 2 个任务需要的算力不足, 所以分配失败了。

时刻 3, 第 1 个计算机仍然正在计算第 1 个任务, 剩余算力不足 3, 所以失败。

时刻 4, 第 1 个计算机仍然正在计算第 1 个任务, 但剩余算力足够, 分配后剩余算力 1。

时刻 5, 第 1 个计算机仍然正在计算第 1, 4 个任务, 剩余算力不足 4, 失败。

时刻 6, 第 1 个计算机仍然正在计算第 4 个任务, 剩余算力足够, 且恰好用完。

【评测用例规模与约定】

对于 20% 的评测用例, $n, m \leq 200$ 。

对于 40% 的评测用例, $n, m \leq 2000$ 。

对于所有评测用例, $1 \leq n, m \leq 2 \times 10^5, 1 \leq a_i, c_i, d_i, v_i \leq 10^9, 1 \leq b_i \leq n$ 。

蓝桥杯 2021 第二轮省赛 A 组 H 题 (B 组 I 题)。

Family Tree B (tree)

P10124 [USACO18OPEN] Family Tree B

【题目描述】

Farmer John 拥有一个传承数代的家族经营的农场，其中的奶牛的根源同样也可以在这个农场中追溯数代。通过检索古老的记录，Farmer John 好奇于现在的这群奶牛互相之间是什么关系。请帮帮他！

【输入格式】

输入的第一行包含 N ($1 \leq N \leq 100$)，之后是两头奶牛的名字。每头奶牛的名字都是由至多 10 个大写字母 (A-Z) 组成的字符串。Farmer John 好奇于这行输入中这两头奶牛之间的关系。

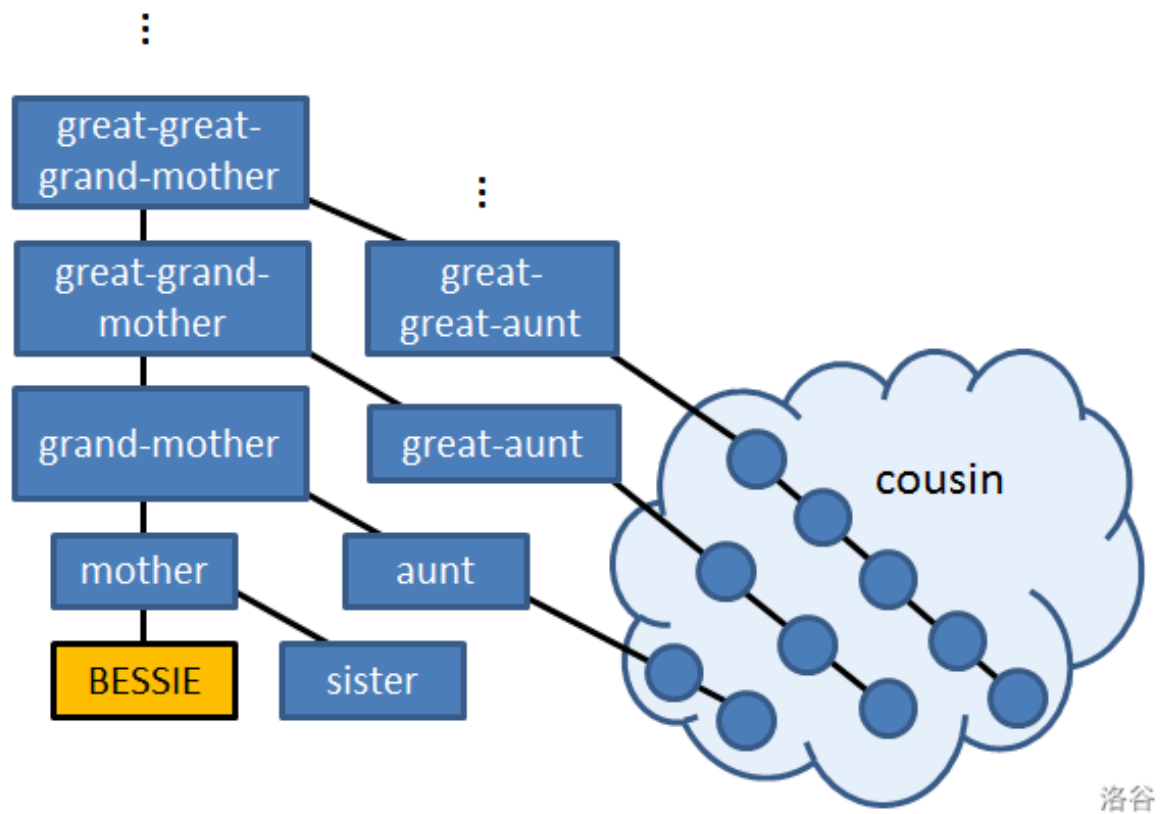
接下来的 N 行，每行包含两头奶牛的名字 X 和 Y ，表示 X 是 Y 的母亲。

【输出格式】

输出包含一行，表示输入第一行指定的两头奶牛之间的关系（简单起见，在下面的例子中，将这两头奶牛称为 BESSIE 和 ELSIE）。下面是可能出现的不同种类的关系：

- 如果 BESSIE 和 ELSIE 的母亲是同一头奶牛，输出 **SIBLINGS**。
- BESSIE 可能是 ELSIE 的直系后代，也就是说 ELSIE 是 BESSIE 的母亲 (**mother**)，外祖母 (**grand-mother**)，外曾外祖母 (**great-grand-mother**)，外曾外曾外祖母 (**great-great-grand-mother**)，等等。如果是这种情况，输出 **ELSIE is the (relation) of BESSIE**，其中 (**relation**) 是适当的关系，比如 **great-great-grand-mother**。
- 如果 ELSIE 不是 BESSIE 的某个祖先或姐妹，但是是 BESSIE 的某个祖先的孩子，那么 ELSIE 就是 BESSIE 的姨母 (**aunt**)。（译者注：英语原题在这里表述有误，供题人已作出声明。）如果 ELSIE 是 BESSIE 的外祖母的孩子，输出 **ELSIE is the aunt of BESSIE**；如果 ELSIE 是 BESSIE 的外曾外祖母的孩子，输出 **ELSIE is the great-aunt of BESSIE**；如果 ELSIE 是 BESSIE 的外曾外曾外祖母的孩子，输出 **ELSIE is the great-great-aunt of BESSIE**；以此类推。
- 如果 BESSIE 和 ELSIE 有任何其他的亲戚关系（也就是说，她们有共同的祖先），她们就是表姐妹，输出 **COUSINS**。
- 如果 BESSIE 和 ELSIE 既没有共同的祖先，其中任何一头也不是另一头的直系后代，输出 **NOT RELATED**。

下图描述了上述关系，你只需考虑这些关系。观察到有一些像是“甥女 (**niece**)”（姊妹的女儿）的关系是不必要的，这是由于如果 BESSIE 是 ELSIE 的甥女，那么 ELSIE 就是 BESSIE 的姨母。



【输入输出样例 #1】

输入 #1

```

1 7 AA BB
2 MOTHER AA
3 GGMOTHER BB
4 MOTHER SISTER
5 GMOTHER MOTHER
6 GMOTHER AUNT
7 AUNT COUSIN
8 GGMOTHER GMOTHER

```

输出 #1

```

1 BB is the great-aunt of AA

```


文件压缩 (zip)

P1124 [ZJOI2001] 文件压缩

【题目背景】

提高文件的压缩率一直是人们追求的目标。近几年有人提出了这样一种算法，它虽然只是单纯地对文件进行重排，本身并不压缩文件，但是经这种算法调整后的文件在大多数情况下都能获得比原来更大的压缩率。

【题目描述】

该算法具体如下：对一个长度为 n 的字符串 S ，首先根据它构造 n 个字符串，其中第 i 个字符串由将 S 的前 $i-1$ 个字符置于末尾得到。然后把这 n 个字符串按照首字符从小到大排序，如果两个字符串的首字符相等，则按照它们在 S 中的位置从小到大排序。排序后的字符串的尾字符可以组成一个新的字符串 S' ，它的长度也是 n ，并且包含了 S 中的每一个字符。最后输出 S' 以及 S 的首字符在 S' 中的位置 p 。

举例： S 是 `example`

1. 构造 n 个字符串。

```
1 example
2 xamplee
3 ampleex
4 mpleexa
5 pleexam
6 leexamp
7 eexampl
```

2. 将字符串排序。

```
1 ampleex
2 example
3 eexampl
4 leexamp
5 mpleexa
6 pleexam
7 xamplee
```

3. 压缩结果。

$S' = \text{xelpame}$, $p = 7$

由于英语单词构造的特殊性，某些字母对出现的频率很高，因此在 S' 中相同的字母有很大几率排在一起，从而提高 S' 的压缩率。虽然这种算法利用了英语单词的特性，然而在实践的过程中，人们发现它几乎适用于所有的文件压缩。

请你编一个程序，读入 S' 和 p ，输出字符串 S 。

保证 S 仅含小写字母（所以输入的 S' 也仅含小写字母）。

【输入格式】

共三行。

第一行是一个整数 n ($1 \leq n \leq 10000$)，代表 S' 的长度。

第二行是字符串 S' 。

第三行是整数 p 。

【输出格式】

一行， S 。

【输入输出样例 #1】

输入 #1

```
1 7
2 xelpame
3 7
4
```

输出 #1

```
1 example
2
```